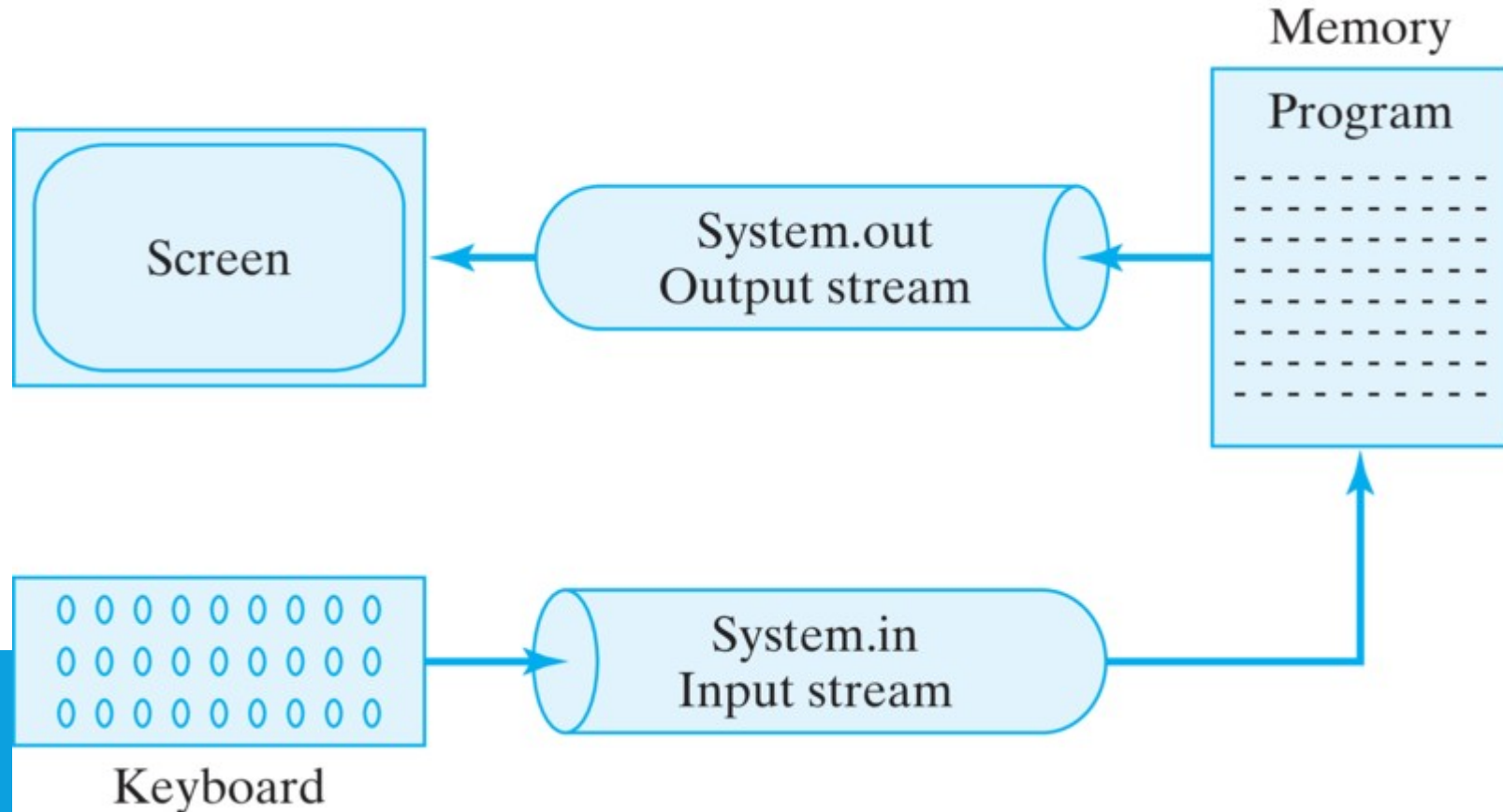


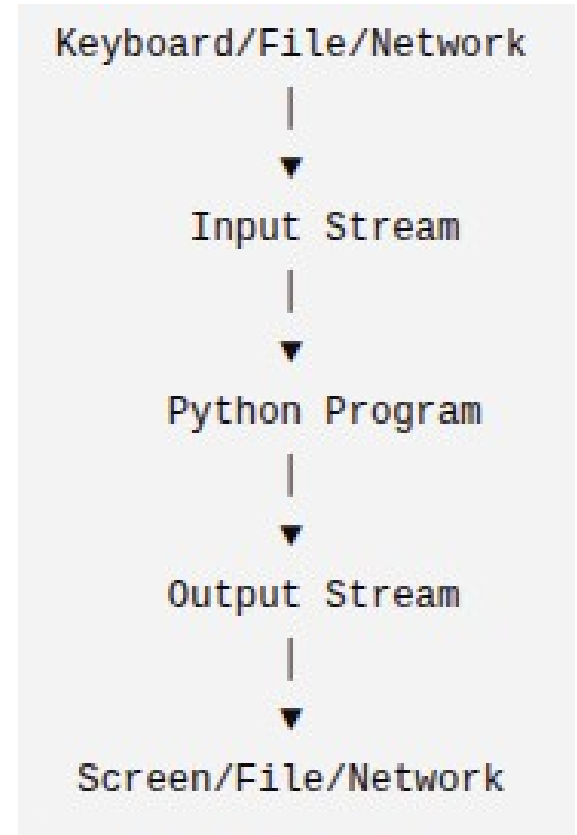
Streams And File Handling



Streams in Python

What are Streams?

- Before learning file handling, we first need to understand Streams, because Python reads and writes every file using streams.
- A stream is simply a flow of data.
- Think of a stream as a road or pipeline through which data travels from one place to another.
- A stream is a continuous flow of data between a source and a destination.
- Whenever data enters a program or leaves a program, it travels through a stream.



Why do we need Streams?

Suppose you type your name.

How does Python receive it?

The data travels from

- Keyboard → Input Stream → Python Program

Similarly,

When Python prints something

The output travels from

- Python Program → Output Stream → Screen

Everything entering or leaving Python goes through streams.

Types of Streams

Python mainly uses three streams.

1. Input Stream
2. Output Stream
3. Error Stream

Input Stream

An Input Stream carries data into the program.
The program receives data from outside.

Example

```
name = input("Enter your name: ")
```

Flow

- Keyboard → your name → Input Stream
→ Python → Variable "name"

Notice that input() always return a string

Output Stream

Output Stream carries data out of the program.

Example

```
print("hello")
```

Flow

- Python (hello) → output stream → monitor

Error Stream

Error Stream carries error messages generated by the program. Instead of normal output, Python sends exceptions to the Error Stream.

Example → `print(10/0)`

Python → Error Generated → Error Stream → Terminal

Notice Nothing is printed through Output Stream. Instead, Python sends the error through Error Stream.

How Data Travels Inside Streams

Many people think streams carry text. Actually, Streams carry bytes. Computers only understand (0,1)
Everything is stored as bytes.

Example → “Hello”

- It is stored as bytes. → H e l l o
- H = 72, e = 101, l = 108, l = 108, o = 111
- 01001000 01100101 01101100 01101100 01101111

Internally when we type hello the keyboard sends bytes these bytes travel from keyboard to python program using input stream python convert them into a python object when python prints python again converts them into bytes that travel from output stream and visible in screen.

This conversion happens automatically.

Types of Streams

There are mainly two types of streams based on the type of data they carry.

Text Stream → Works with text (characters). Data type → str

Binary Stream → Works with raw bytes. Data type → bytes (Used for images, videos, audio, pdf, executable files.)

Uses of Streams

Streams are used almost everywhere in computing.

- 1. Web Servers → When a browser sends a request.
- **2. Large File Handling** → Without streams, Python would load the entire file into memory, which is inefficient for very large files. (Data read in small chunks)
- **Network Programming** → When two computers communicate, data travels as streams. This is how chat applications, video calls, online games, and APIs exchange data.

Buffering in Python

Buffering

Before learning file handling, we must understand Buffering, because Python uses buffering automatically whenever it reads from or writes to files. Without buffering, file operations would be extremely slow.

What is Buffering?

Buffering is the process of temporarily storing data in memory (RAM) before it is processed or transferred to its final destination.

Instead of reading or writing one byte at a time, Python first stores data in a buffer and then processes it in larger chunks.

A buffer is a temporary storage area in RAM used to make input and output operations faster.

Eg → bucket water.

Without Buffer read 1 byte per visit The disk is accessed repeatedly that makes it Very slow.

With Buffer read 4096 bytes buffer only one disk access is required much faster.

Why does buffering exist?

Computer have different speeds.

Device	Speed
CPU	Extremely Fast
RAM	Very Fast
SSD	Fast
HDD	Slow
Network	Slower

The CPU can process data millions of times faster than a hard disk.

If the CPU had to wait for the disk every time it needed one byte, the program would become very slow.

Buffering solves this problem.

Problems Without Buffering

1. Too Many Disk Accesses
2. Too Many System Calls
3. Slow Performance

Types of Buffering

1. Unbuffered

In unbuffered mode, Python does not use a buffer. Every read or write goes directly to the device.

Only valid for binary mode. → `open('file_name', buffering=0)`

- **Advantages**
 - Immediate reading/writing
 - No waiting
- **Disadvantages**
 - Very slow
 - Too many system calls
 - Rarely used

2. Line Buffered

Data is stored in the buffer until a newline (`\n`) is encountered. When Python sees a newline, it flushes the buffer automatically.

Eg → `open('file_name', buffering=1)`

Used In

- Terminal Output
- Logs
- Console Programs

3. Fully Buffered

Python stores data until the buffer becomes full. Then it writes everything together. This is the default mode for most file operations.

Eg → `open('file_name', 'r')`

Python may use 4096 Bytes or 8192 Bytes depending on the operating system.

Advantages

- Fastest
- Fewest disk accesses
- Default mode

Read Buffer vs Write Buffer

Buffering happens during both reading and writing.

Read Buffer

Suppose → `f.read(1)`

Read 4096 Bytes and store in RAM but return only 1 byte the remaining bytes stay inside RAM. Next `read()` does not go to disk it uses RAM.

Write Buffer

Suppose → `f.write("Hello")`

Python does not immediately write to the disk. Instead it write on buffer when buffer becomes full then `flush()` is called and then `close()` is called then python writes everything.

Flush in Python

In the previous chapter, we learned that Python uses buffering to improve performance. Because of buffering, data is not always written to the file immediately.

This leads us to an important concept called Flush.

Why Do We Need Flush?

Suppose you write data to a file. `f.write("hello")`

The data first goes into the buffer (RAM), not directly to the file. If the program crashes before the buffer is written to disk, the data may be lost.

To prevent this, Python provides the `flush()` method.

What is Flush?

`flush()` forces Python to write all buffered data immediately from RAM to the actual file on disk.

```
f = open("data.txt", "w")  
  
f.write("Hello")
```

At this point,

the file may still be empty because the data is only in the buffer.

```
f = open("data.txt", "w")  
  
f.write("Hello")  
  
f.flush()
```

Now the file definitely contains

Without flush suppose before calling `f.flush()` the computer suddenly shuts down the data never reaches the disk. With flush even if the computer crashes the file already contains the data. `Flush()` does not close the file it only writes buffered data to disk. Don't flush every time it forces to disk write.

`Close()` automatically flushes when you close a file.

When Should You Use flush()?

You should use flush() when data must be saved immediately.

1. Logging
2. Long Running Programs
3. Real-Time Output → `print("Loading...", end="", flush=True)`